



Obligatorio Microprocesadores

Ingeniería en Electrónica, Universidad ORT Uruguay

Mateo Menchaca - 251812

Mateo Silva - 282489

Docente: Ismael Garrido

Junio, 2025

Índice

1	Introducción	5
1.1	Estructura basica del obligatorio	5
1.2	Opcionales	5
2	Hardware	7
2.1	Componentes usados	7
2.1.1	Uc32	7
2.1.2	Pic32	7
2.1.3	SSD1306	7
2.1.4	Switch	7
2.1.5	Buzzer pasivo	8
2.1.6	Esquemático	8
2.1.7	Conexiones SSD1306	9
2.1.8	Conexiones Botones y Buzzer	9
3	Software	10
3.1	Configuración Software	10
3.1.1	Pines	10
3.1.2	SPI	10

3.1.3	UART	12
3.1.4	Configuración SSD1306	12
3.2	Funciones de comunicación	14
3.2.1	Función enviar SPI	14
3.2.2	Función imprimir_bitmap	15
3.3	Funciones adicionales	15
3.3.1	Funcion recibir_nota_btn	15
3.3.2	recibir_UART	16
3.3.3	imprimir_nota	16
3.3.4	Polling	16
3.3.5	Suena Buzzer	17
3.3.6	Almacenar y borrar Row	18
3.4	Guardado en memoria flash	18
4	Módulos y flujo	21
4.1	Diagrama de flujo	21
4.2	Funcion Menu	23
4.3	funcion_piano	24
4.4	funcion_melodia	24
4.5	Módulo grabación	25

5	Manual de Usuario	26
5.0.1	Piano	27
5.0.2	Guardar	27
5.0.3	Reproducir memoria flash	27
5.0.4	Reproducir melodía 1, 2 y 3	28
5.1	UART	28
6	Anexo	29
6.1	Datasheets	29

Introducción

1.1 Estructura basica del obligatorio

Como obligatorio, se nos propuso, haciendo uso de la placa *uc32*, un display monocromático de 128x64 píxeles y un parlante, crear un piano digital.

El mismo debe ser capaz de reproducir las siete notas musicales naturales (Do, Re, Mi, Fa, Sol, La, Si), mostrando en el display cuál nota está siendo ejecutada. La funcionalidad debe consistir en la reproducción de una nota a la vez, con una duración fija e igual para todas ellas.

1.2 Opcionales

Además de esta funcionalidad básica, se propusieron ciertos opcionales, mediante los cuales debía alcanzarse un total de 30 puntos para completar el máximo puntaje posible.

En nuestro caso, implementamos los siguientes opcionales:

- **Sostenidos:** Se agregan las notas sostenidas (Do#, Re#, Fa#, Sol#, La#).
- **Reproductor:** El piano ofrece un menú con distintas melodías, permitiendo seleccionar una para su reproducción. Se incluyen al menos tres melodías diferentes.
- **Grabación de melodía:** Se permite grabar una melodía y luego reproducirla.
- **Visualización del piano:** El display muestra un teclado, y al presionar una tecla (o reproducirla por otro medio), esta se resalta visualmente.
- **Persistencia de grabación:** La melodía grabada se mantiene almacenada incluso al reiniciar el dispositivo.

- **Modo instrumento:** Se permite recibir comandos por puerto serie para tocar el piano. Estos comandos simulan la presión de teclas reales, incluyendo su visualización si está habilitada.

Las teclas correspondientes son: a=Do, s=Re, d=Mí, f=Fa, g=Sol, h=La, j=Si, w=Do#, e=Re#, t=Fa#, y=Sol#, u=La#.

Con la suma de estos opcionales, junto a la estructura básica del proyecto y su respectiva documentación, se alcanza el puntaje total propuesto por el obligatorio.

Hardware

2.1 Componentes usados

2.1.1 Uc32

La uc32 es una placa de desarrollo basada en el microcontrolador PIC32MX340F512H. Ofrece 43 pines de entrada y salida tanto analógicos como digitales, puede ser operada a 3,3V y proporciona acceso a las funciones periféricas del microcontrolador.

2.1.2 Pic32

El PIC32 es una familia de microcontroladores de 32 bits, basada en la arquitectura MIPS. Estos microcontroladores ofrecen altas prestaciones en procesamiento y múltiples periféricos. Asimismo el Pickit 3 es una herramienta utilizada para depurar y programar estos microcontroladores en el IDE MPLAB.

2.1.3 SSD1306

El SSD1306 es un controlador gráfico para pantallas OLED monocromáticas, comúnmente usado en módulos de 128×64 píxeles. Se comunica con microcontroladores a través de interfaces como SPI o I²C, permitiendo el control individual de cada píxel.

2.1.4 Switch

Los switches utilizados en este proyecto están conectados mediante una configuración con resistencias pull-down, entonces, cuando no se está presionando el botón, el pin de entrada del microcontrolador se mantiene en low.

2.1.5 Buzzer pasivo

El buzzer pasivo, tiene como requerimiento una señal de entrada de tipo PWM para producir sonido.

2.1.6 Esquemático

Las resistencias utilizadas son de $10\text{ k}\Omega$

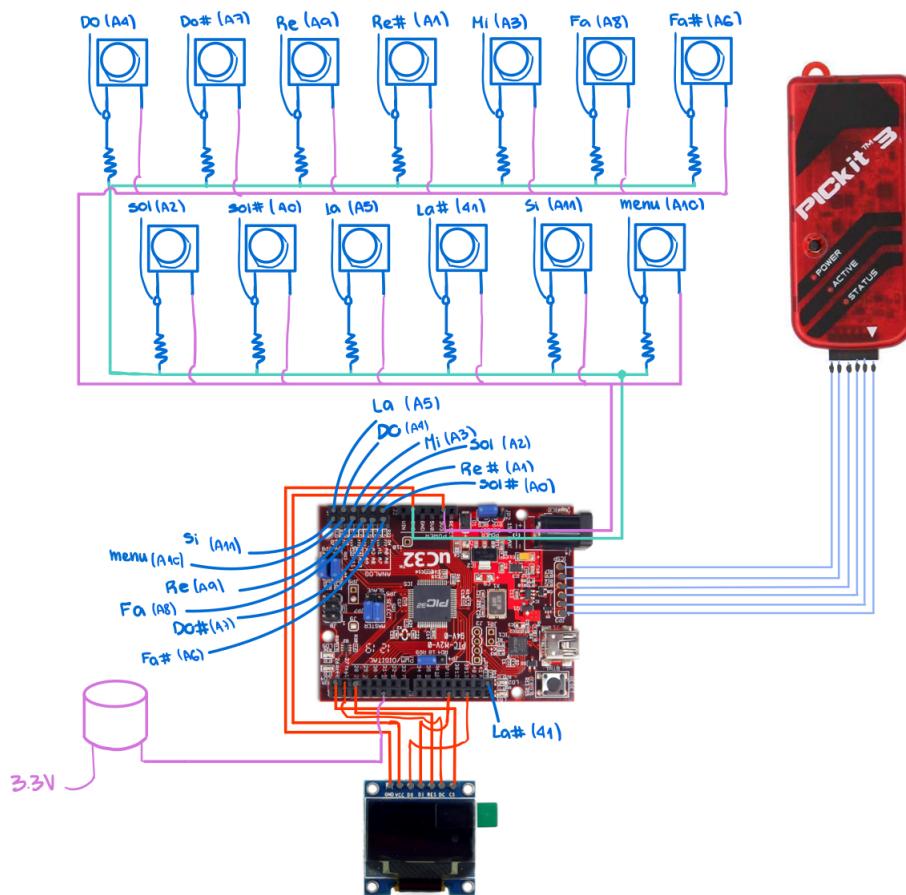


Figure 2.1: Esquemático

2.1.7 Conexiones SSD1306

- GND → GND
- VCC → 3.3V
- D0 → 13
- D1 → 11
- RES → 28
- DC → 27
- CS → 26

2.1.8 Conexiones Botones y Buzzer

- Do → A4
- Re → A9
- Mi → A3
- Fa → A8
- Sol → A2
- La → A5
- Si → A11
- Do# → A7
- Re# → A1
- Fa# → A8
- Sol# → A0
- La# → 41
- Menú → A10
- Buzzer → 5

En las conexiones, a la izquierda se puede ver que esta siendo conectado y a la derecha en que pin (los pines utilizados son los de la Uc32).

Software

3.1 Configuración Software

Dentro de la configuración de software se implementan funciones pensadas para configurar las funcionalidades del sistema. Dentro de esta sección se detallan las secuencias de inicializaciones generales, ejecutadas previas al bucle principal y por fuera del mismo.

3.1.1 Pines

Se definen las salidas y entradas seteando los TRIS correspondientes, se setean como pines digitales todos los pertenecientes a PORTB dado que los mismos tienen una naturaleza de analógicos/digitales. De esta manera se establecen los pines utilizados de la placa.

3.1.2 SPI

Se implementa una comunicación sincrónica serial mediante el protocolo 4-Wire SPI, donde el PIC32 actúa como maestro. La comunicación opera a 50 kHz, con parámetros $CPOL = 0$ y $CPHA = 0$, lo que implica que el dato se envía en el flanco descendente del reloj.

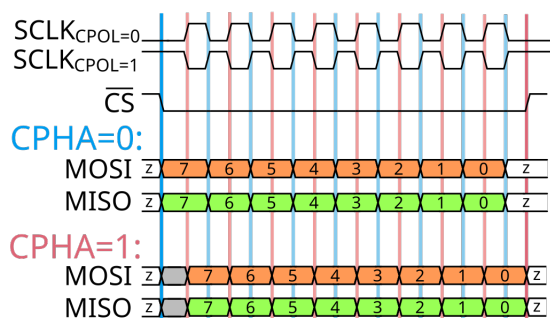


Figure 3.1: Timing SPI

Inicializar SPI

Para inicializar SPI hay que seguir los pasos de la datasheet:

- Deshabilitar interrupciones SPI en el registro IECx correspondiente.
- Detener y reiniciar el módulo SPI desactivando el bit ON.
- Limpiar búfer de recepción.
- Limpiar bit ENHBUF (SPIxCON<16>) si se utiliza el modo de buffer estándar o establecer bit si usa el modo de enhanced buffer.
- Si no se utilizan interrupciones SPI, omitir este paso y continuar con el paso 6. De lo contrario, realizar los siguientes pasos adicionales:
 - a) Limpiar las flags de interrupción SPIx en el registro IFSx correspondiente.
 - b) Escribir los bits de prioridad y subprioridad de interrupción SPIx en el registro
 - c) Establecer los bits de habilitación de interrupción SPIx en el registro IECx correspondiente.
- Escribir el registro de la tasa de baudios, SPIxBRG.
- Limpiar el bit SPIROV (SPIxSTAT<6>).
- Escribir la configuración deseada en el registro SPIxCON con MSTEN (SPIxCON<5>)=1.
- Habilitar la operación SPI estableciendo el bit ON (SPIxCON<15>).
- Escribir los datos a transmitir en el registro SPIxBUF. La transmisión (y recepción) comenzará tan pronto como los datos se escriban en el registro SPIxBUF.

En nuestro caso no usamos interrupciones, por lo tanto salteamos los pasos que incluyen estas. Cabe recalcar que estos pasos tienen que realizarse en el orden que fueron escritos, de lo contrario, la configuración no será realizada correctamente

3.1.3 UART

Para comunicación asíncrona configuramos y habilitamos el UART en modo velocidad estándar para que se comunique a 1200 baudios con recepción de datos.

3.1.4 Configuración SSD1306

La pantalla oled SSD1306 requiere de una configuración secuencial precisa y extensa donde se define como se va a utilizar y donde se realiza la secuencia de encendido. Para esto se envían comandos vía SPI, manejando las señales de control para la correcta interpretación de los datos. Se habilita la alimentación interna, se define el brillo y contraste de la pantalla, se establece la relación de multiplexación, se define el inicio de la pantalla, offset, posibles re-mapeados de los segmentos y dirección de escaneo de la misma, se define la frecuencia del oscilador (370 kHz) y finalmente se enciende. Para esto, hicimos uso de la función `enviarSPI`, para enviar por este sistema de comunicación los comandos, para configurar la pantalla de la manera indicada:

La secuencia de encendido recomendada comienza con la activación de la alimentación lógica (VDD). Una vez que esta tensión se estabiliza, se debe llevar el pin de reset (RES#) a nivel bajo durante al menos 3 us y luego pasarlo a nivel alto. Posteriormente, se espera nuevamente un mínimo de 3 us con el pin RES en nivel bajo antes de continuar. Luego de esto se realiza (mínimamente) la siguiente configuración para terminar de encender la pantalla

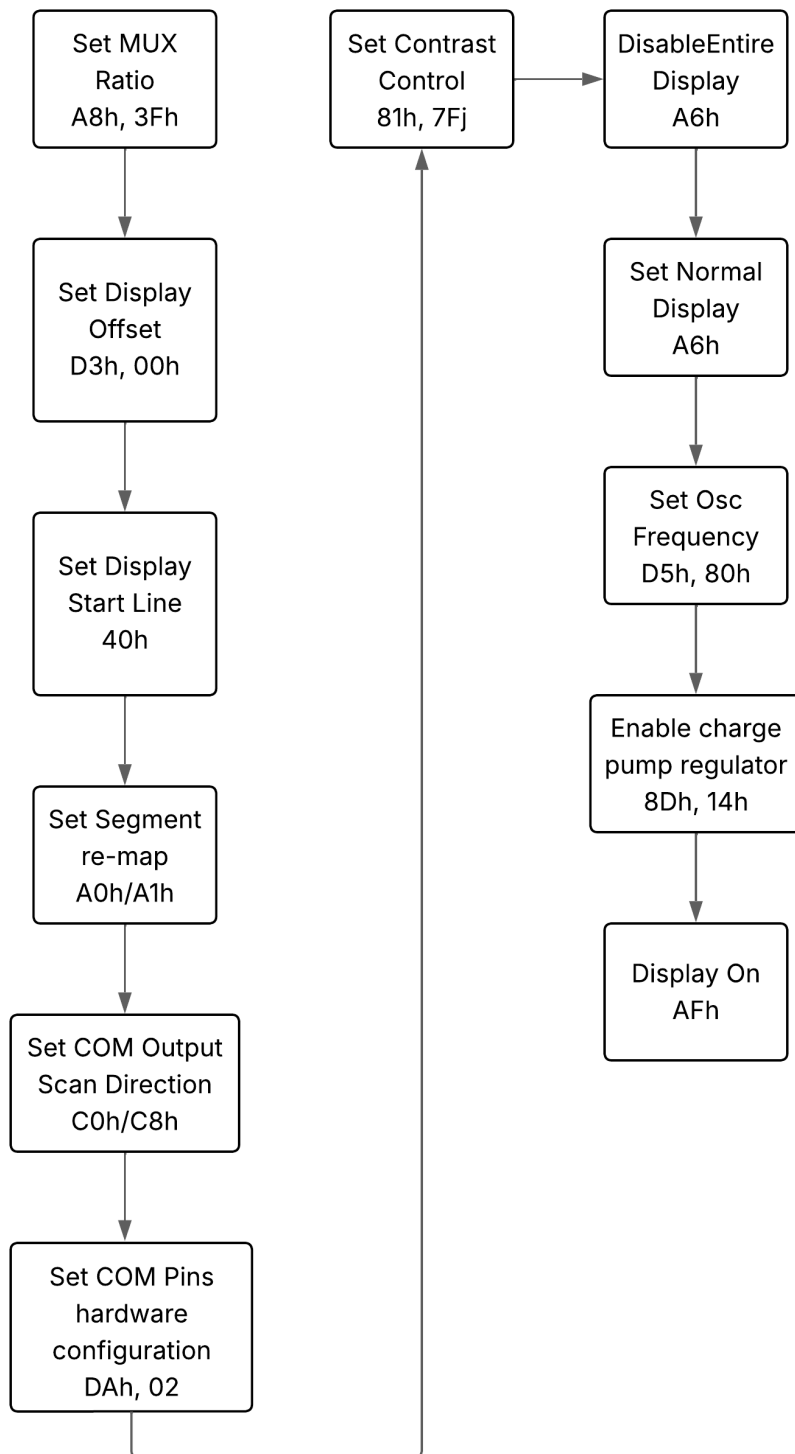


Figure 3.2: Inicializacion de SSD1306

3.2 Funciones de comunicación

En esta sección se encuentran implementadas las funciones que permiten utilizar la pantalla oled, con dos funciones fundamentales donde se realiza la comunicación vía SPI y donde se definen los bytes de datos para mostrar una imagen en la pantalla.

3.2.1 Función enviar SPI

Esta función implementa la comunicación entre la placa y la pantalla SSD1306, para esto se encarga de manejar y actualizar las señales de control necesarias para manejar la pantalla, en conjunto con el protocolo SPI implementado, suministrando el reloj(SCLK) necesario para lograr la comunicación sincrónica enviando paquetes de 8 bits(SDIN).

Pin Name Bus Interface	Data/Command Interface								Control Signal				
	D7	D6	D5	D4	D3	D2	D1	D0	E	R/W#	CS#	D/C#	RES#
8-bit 8080	D[7:0]								RD#	WR#	CS#	D/C#	RES#
8-bit 6800	D[7:0]								E	R/W#	CS#	D/C#	RES#
3-wire SPI	Tie LOW					NC	SDIN	SCLK	Tie LOW		CS#	Tie LOW	RES#
4-wire SPI	Tie LOW					NC	SDIN	SCLK	Tie LOW		CS#	D/C#	RES#
I ² C	Tie LOW					SDA _{OUT}	SDA _{IN}	SCL	Tie LOW			SA0	RES#

Figure 3.3: Señales de SSD1306

Se reciben como parámetros el byte a ser enviado y los valores deseados de las señales de control, Chip selector(CS) al estar bajo habilita la escritura, Data/Comando(D/C) se coloca low para procesar la data como comando y high para procesarla como dato, resett(RES) al bajar este pin a low se desencadena la inicialización del chip.

Al finalizar cada envío de data se setea la señal CS en high, por lo que para ejecutar acciones que usen comandos múltiples se utiliza el parámetro comando doble para mantener CS low.

3.2.2 Función imprimir_bitmap

Esta función se encarga de imprimir una imagen (bitmap) en la pantalla SSD1306 utilizando comunicación SPI. Recibe como parámetro en \$a0 la dirección de la imagen que se desea mostrar. La función define el ancho de la pantalla como 128 columnas y calcula que hay 8 páginas verticales, dado que cada página representa 8 píxeles de alto ($64 / 8 = 8$).

El procedimiento consiste en recorrer cada una de las 8 páginas (filas de 8 píxeles), y dentro de cada página, enviar los 128 bytes correspondientes a cada columna. Para cada página se configuran los comandos necesarios: número de página, columna baja y columna alta. Luego, se envían 128 bytes de datos (uno por cada columna) desde la imagen.

Cada byte enviado representa los 8 bits verticales de una misma columna en la página actual. El proceso se repite para cada página, hasta que se completa toda la pantalla.

3.3 Funciones adicionales

3.3.1 Funcion recibir_notas_btn

Esta función detecta qué nota musical fue presionada a partir de una lectura de los botones conectados al PORTB. Se recibe en \$a0 un valor donde ya se han filtrado los bits irrelevantes (es decir, solo están en 1 los bits correspondientes a los botones activos).

Luego, se aplica una serie de máscaras bit a bit para identificar qué botón está siendo presionado. Para cada posible nota, se compara con una máscara específica, y si hay coincidencia (es decir, el bit correspondiente está en 1), se asigna a \$t3 el valor ASCII asociado a esa nota (por ejemplo, 'a' para Do, 's' para Re, etc.).

Finalmente, si se detecta alguna nota, se salta a una etiqueta, donde se copia ese valor a \$v0. Si no se presiona ningún botón, la función retorna sin modificar \$v0.

3.3.2 recibir_UART

Esta función recibe el comando enviado por UART en \$a0. Se branchea si se recibió alguna nota valida, si no se recibe nada devuelve 1 (valor asignado porque si sale 0 vuelve al menú, por lo que si no devuelve un valor distinto de 0 estaría volviendo al menú cada vez que se hace el polling y no se toco nada). Al branchearse, se mueve a \$v0, el valor de la nota enviada por UART.

3.3.3 imprimir_nota

Esta función se encarga de asignar la imagen y divisor de frecuencia, según la nota que haya sido presionada. Recibe por \$a0, el ascii con la nota tocada, en base a eso branchea según lo que se haya seleccionado, carga la imagen y el divisor correspondiente, para devolver en \$v0 y \$v1 respectivamente.

3.3.4 Polling

Esta función realiza un loop de polling hasta que se recibe una nota, ya sea por UART o por los botones conectados a PORTB. Si llega un dato por UART, se detecta el comando recibido, se espera a que el transmisor esté listo y luego se reenvía ese dato por el registro TX. A continuación, se interpreta el valor recibido para determinar qué nota representa y se devuelve en \$v0. Si en cambio se detecta una entrada por PORTB, se filtran los bits relevantes y se identifica la nota correspondiente al botón presionado, esto se devuelve en \$v0.

3.3.5 Suena Buzzer

Para generar en el buzzer la señal necesaria para hacer sonar cada nota, implementamos esta función con la que generamos una salida con PWM de 50% duty cycle 6.1 con un período variable, de forma de poder generar cada frecuencias especificas. Para generar cada frecuencia se utiliza el divisor de frecuencia asignado a cada nota, el cual se recibe como parámetro calculado previamente.

$$\text{Período del PWM} = (PR + 1) \cdot TPB \cdot \text{Prescaler}$$

$$\text{Frecuencia del PWM} = \frac{1}{\text{Período del PWM}}$$

Nota	Frecuencia (Hz)	Divisor frecuencia
Do	261.6	1910
Do#	277.2	1802
Re	293.7	1701
Re#	311.1	1606
Mi	329.6	1515
Fa	349.2	1430
Fa#	370.0	1350
Sol	392.0	1274
Sol#	415.3	1202
La	440.0	1135
La#	466.2	1071
Si	493.9	1011

Table 3.1: Notas musicales con su frecuencia y divisor correspondiente para PR2

Para generar el PWM usamos la funcionalidad del Pic de output compare junto con otra funcionalidad Timer2. El Timer2 lo usamos como un contador de 16 bits, sin interrupciones, en modo estándar y con un divisor 1:1, de forma que tiene una frecuencia 500 kHz. La señal PWM se mantiene encendida con un delay para que la nota tenga una duración, pasado este delay se apaga el PWM y se resetea a 0 el timer.

3.3.6 Almacenar y borrar Row

Estas dos funciones manejan el contenido del array de tamaño Row utilizado para almacenar una melodía en flash. Borrar Row borra completamente el array almacenando 0 en toda su extensión. Por otro lado Almacenar nota Row guarda una nota recibida como parámetro en la primer posición disponible en el array hasta completar los 512 bytes de capacidad.

3.4 Guardado en memoria flash

La memoria Flash es un tipo de memoria persistente, por lo que nos permite guardar una melodía capaz de sobrevivir al apagado de la placa. Se programa la memoria en Run-Time Self-Programming (RTSP). La memoria dentro de esta sección se divide en word, row(128 words) y page(8 rows). Las acciones a realizar son Borrar y Guardar, para guardar utilizamos una row y para borrar usamos la mínima expresión borrrable, una page. Para modificar la memoria flash se programa una secuencia específica donde se determina el tipo de operación a realizar, las direcciones del contenido en caso de ser necesario y una secuencia de desbloqueo la cual existe para evitar accesos indeseados a flash.

Para guardar una Row la secuencia es la siguiente:

1. Cargue la fila completa de datos que se programará en la memoria **SRAM** del sistema . La dirección de origen debe estar **alineada a palabra** .
2. Establezca el registro **NVMADDR** con la dirección de inicio de la fila **Flash** que se va a programar . Es crucial que esta dirección sea la **dirección física** de la memoria Flash, y no la dirección virtual.
3. Establezca el registro **NVMSRCADDR** con la **dirección física de origen** del búfer de datos en la **SRAM** del paso 1 .
4. Suspender o deshabilitar todos los iniciadores que puedan acceder al bus periférico, como el DMA o las interrupciones.

5. Establecer el bit **WREN** (**NVMCFG<14>**) para permitir escrituras, y configurar el campo **NVMOP<3:0>** con el código correspondiente a la operación de escritura de fila (*Row Program*).
6. Esperar a que se establezca el detector de bajo voltaje (**LVD**).
7. Cargar el valor **0xAA996655** en un registro general del CPU (por ejemplo, **X**).
8. Cargar el valor **0x556699AA** en otro registro general (por ejemplo, **Y**).
9. Cargar el valor **0x00000080** en un tercer registro (por ejemplo, **Z**).
10. Escribir el valor de **X** en el registro **NVMKEY**.
11. Escribir el valor de **Y** en **NVMKEY**.
12. Escribir el valor de **Z** en **NVMCONSET**.
13. Esperar que el bit **WR** (**NVMCFG<15>**) se borre automáticamente por hardware, lo que indica que la secuencia de programación ha finalizado.
14. Limpiar el bit **WREN** (**NVMCFG<14>**).

Durante el proceso, al activarse el bit **WR**, la secuencia de programación comienza y el CPU no puede ejecutar código desde la memoria Flash hasta que dicha secuencia finaliza. Además se desactivan las interrupciones dado que si ocurre una se cancela el proceso.

Se tiene que considerar que la dirección de destino de la row en el espacio flash que se sepa vacía, dado que se puede superponer con secuencias de boot o de debug propias del pic. Para borrar se realizan los mismos pasos a diferencia de 1-3, donde se debe hacer en su lugar los siguientes.

1. Configurar el registro **NVMADDR** con la dirección de la página que se desea borrar. El controlador ignora los bits menos significativos al seleccionar la página.
2. Ejecutar la secuencia de desbloqueo utilizando el comando de *Page Erase*, configurando el campo **NVMOP<3:0>** con el código correspondiente.

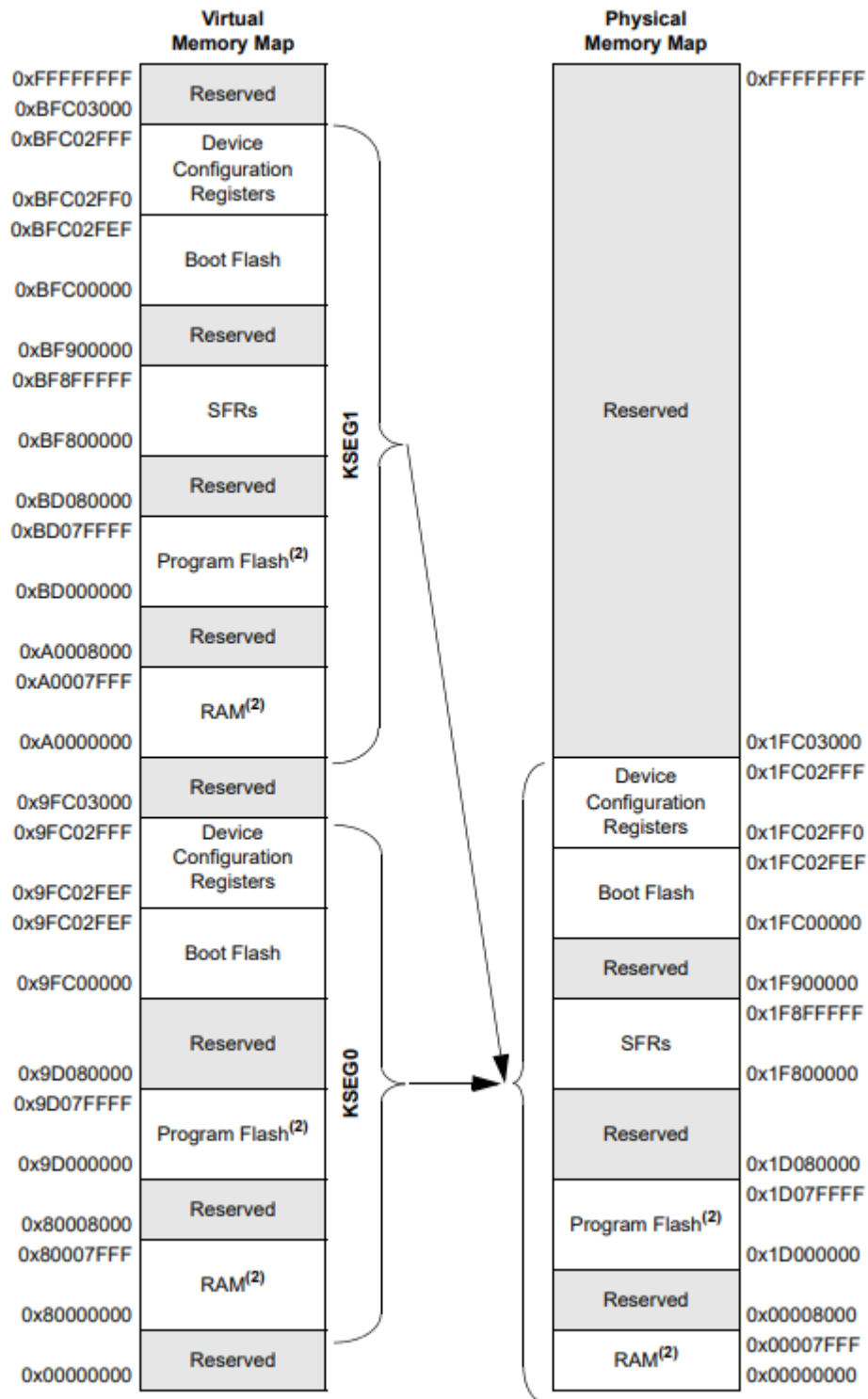


Figure 3.4: Mapa memory

Módulos y flujo

4.1 Diagrama de flujo

Una vez booteado, el sistema ejecuta una secuencia de pasos única de configuración preparando los periféricos y sus funcionalidades para su uso. Posteriormente entra en un bucle donde ocurren las implementaciones del proyecto. Durante esta etapa el sistema se encuentra en su módulo menú, donde se relaciona un registro con la posición indicada al usuario por la interfaz, permitiendo acceder a cada acción disponible, las cuales una vez finalizan vuelven a ejecutar el bucle del sistema. El siguiente diagrama de flujo muestra esta dinámica de forma general, y a continuación se desarrollarán cada uno de los módulos involucrados.

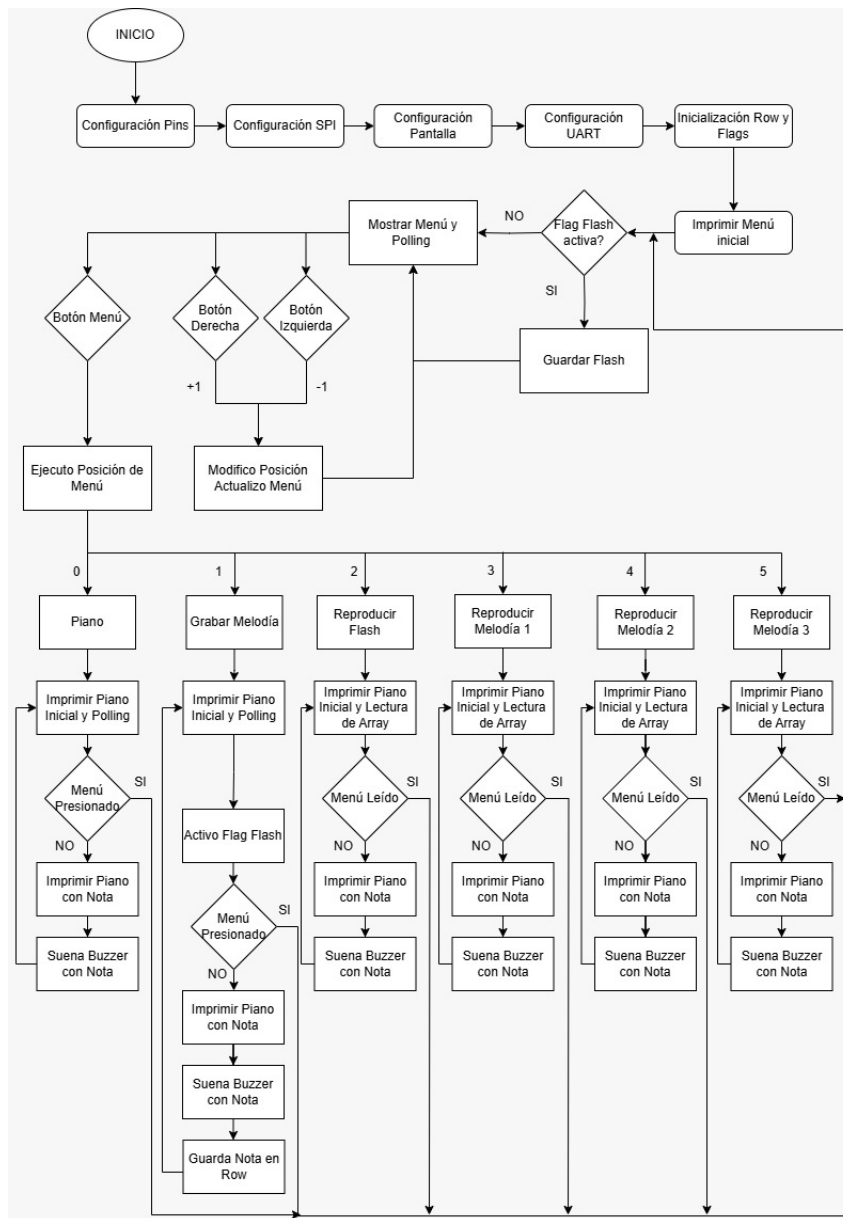


Figure 4.1: Diagrama de flujo

4.2 Funcion Menu

Esta función se encarga de controlar la navegación dentro de un menú, recibiendo en el registro `$s0` la posición actual en pantalla. Mediante un bucle que utiliza la función `Polling`, se espera la entrada de un botón válido. Se consideran válidas las teclas 'u' (para moverse hacia la izquierda, botón LA#), 'j' (para moverse hacia la derecha, botón SI) y el valor 0 (para aceptar o seleccionar la opción actual, botón menú). Mientras no se reciba una de estas teclas, el bucle continúa esperando nuevas lecturas. Una vez detectada una entrada válida, se preparan los argumentos correspondientes en `$a0` y `$a1`, y se llama a la función `accionar_menu`.

Accionar menú gestiona el comportamiento visual del menú en pantalla. Recibe en `$a0` el comando ingresado (izquierda, derecha o aceptar) y en `$a1` la posición actual del menú. Primero se verifica si se debe mover de posición o derivar a una funcionalidad. En caso de que se toque izquierda o derecha, se suma o se resta al registro contador siempre y mientras no exceda los límites del menú, actualizando la pantalla con la nueva posición. En caso de presionar aceptar, se deriva a una serie de branches que determinan qué funcionalidad se pretende accionar según la posición del menú, para proceder a ejecutar dicho módulo.

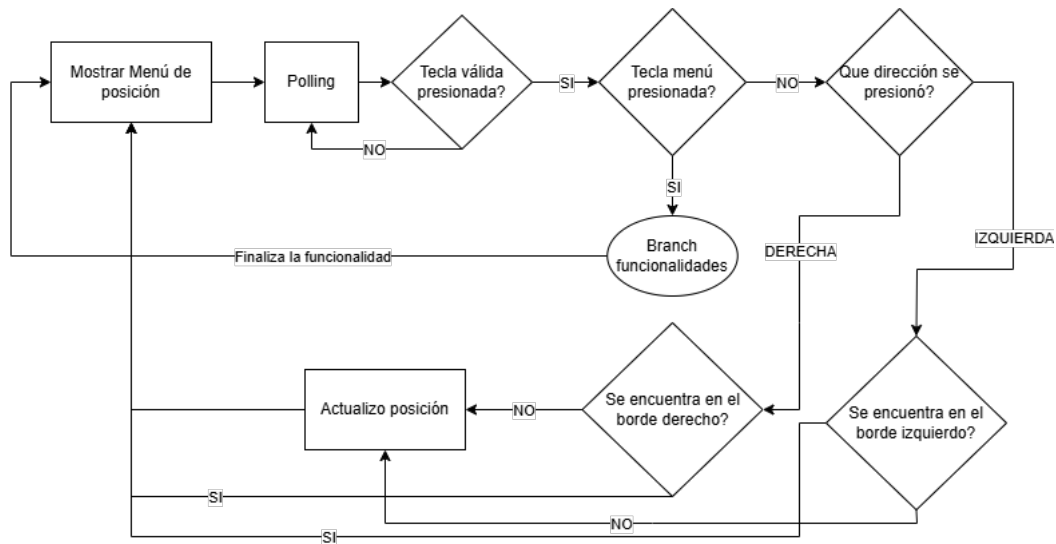


Figure 4.2: Diagrama función menú

4.3 funcion_piano

Esta función ejecuta la lógica de la aplicación piano, que incluye las notas obligatorias y sostenidos opcionales, junto con una animación que indica la tecla presionada. En cada iteración del bucle principal, se imprime la imagen base del piano con `imprimir_bitmap`, luego se espera una nota por `Polling`. Si se detecta una nota (diferente de 0), se determina cuál es, se imprime visualmente con `imprimir_nota`, y luego se reproduce la frecuencia correspondiente con `Suena_Buzzer`. El bucle continúa hasta que se presiona el botón de menú, lo cual devuelve 0 y finaliza la función.

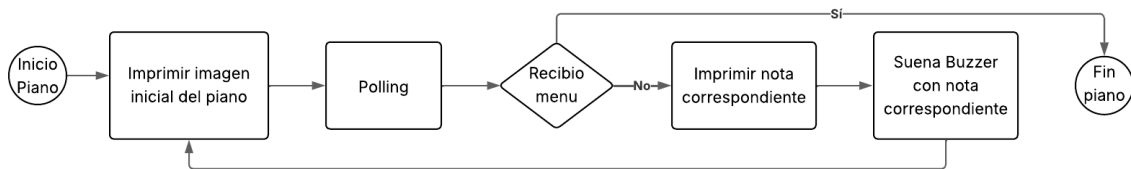


Figure 4.3: Diagrama función piano

4.4 funcion_melodia

Esta función reproduce una melodía a partir de la dirección que recibe en `$a0`. Recorre el array de notas una por una, verificando que cada valor sea distinto de cero (lo cual indica el final de la melodía). Para cada nota, llama a `imprimir_nota`, que devuelve la imagen correspondiente y la frecuencia del buzzer. Luego, se imprime la imagen con `imprimir_bitmap` y se reproduce el sonido con `Suena_Buzzer`. El proceso se repite hasta alcanzar el final del array.

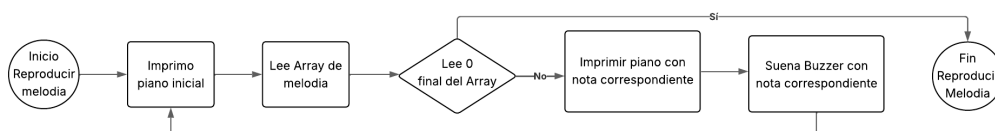


Figure 4.4: Diagrama función melodía

4.5 Módulo grabación

Este módulo contiene la lógica para producir una melodía y guardarla en la memoria flash. Una vez iniciado ejecuta `Clear_Flash` y `BorrarRow`, preparando el espacio en la memoria flash y en el espacio de Row, luego ingresa en el loop donde imprime el piano inicial y hace `polling` hasta recibir una nota, mientras no recibe menú actualiza la pantalla con `imprimir_bitmap` y reproduce la nota con `Suena_Buzzer`, finalmente guarda la nota en la Row con `Almacenar en Row` y vuelve a iniciar la secuencia. Cuando detecta que se ingreso menú abandona la secuencia y vuelve al loop principal del programa donde se ejecuta `Guardar_Flash`, que guarda en flash la Row recién generada y por último desactiva la Flag de Flash.

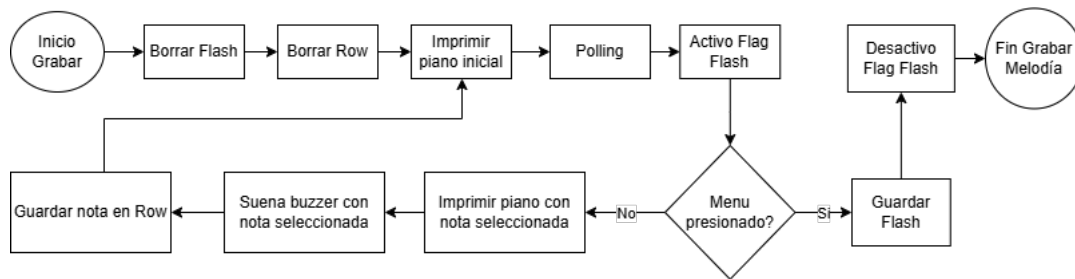


Figure 4.5: Diagrama función grabar melodía

Manual de Usuario

A continuación se va a poder apreciar un diagrama de lo que serian los botones disponibles para usar en el proyecto.

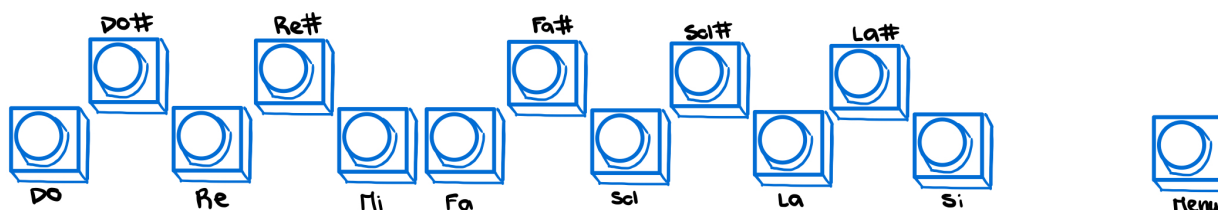


Figure 5.1: Disposicion Proto Board

Una vez encendida la placa se va a cargar el siguiente menú, donde se pueden apreciar las primeras aplicaciones del proyecto.

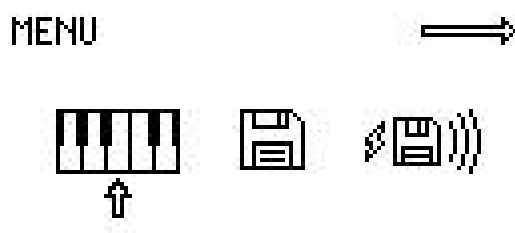


Figure 5.2: Primer pagina menú

Donde se pueden realizar movimientos, a la izquierda con la tecla La# y a la derecha con la tecla Si. Si se desea entrar a una aplicación, se mueve la posición a la aplicación que se desee y se presiona el botón menú.

Este proyecto cuenta con 6 aplicaciones en total, por lo tanto, si se desea ver las demás hay que mover a la derecha hasta llegar a la siguiente pagina.

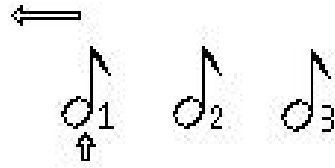


Figure 5.3: Segunda pagina menú

Una vez dentro de la aplicación deseada, el funcionamiento de las teclas cambiara.

5.0.1 Piano

En esta aplicación se va a poder apreciar un piano donde cada botón representa una nota, como esta escrito en el primer diagrama. Aquí se podrán tocar 13 notas musicales, esta cuenta con sus notas comunes y sostenidos. Solo se puede tocar una nota a la vez.

Si se desea salir de la aplicación, se toca la tecla menú en el momento que desee.

5.0.2 Guardar

En esta aplicación se podrá guardar una melodía, con una máximo de 512 notas. Para terminar la melodía se presiona el botón menú el cual te lleva al menú principal y guarda la melodía.

5.0.3 Reproducir memoria flash

Para reproducir memoria flash, primero se debe guardar una melodía en la aplicación guardar melodía. Si no se hace esto, la aplicación no funciona de manera correcta. Una vez guardada una melodía, se comienza a reproducir en el piano, esta no puede ser cortada, el programa te devuelve al menú principal una vez se termina de reproducir. La melodía persiste al apagado de la placa, por lo que una vez se haya guardado una melodía no es necesario guardar nuevamente al encenderla de nuevo.

5.0.4 Reproducir melodía 1, 2 y 3

Reproduce una melodía precargada en el programa, esta no puede ser cortada, por lo tanto, no se puede volver al menú mientras se esta reproduciendo. Al terminar la melodía, el programa te devuelve automáticamente al menú principal.

Las melodías disponibles son las siguientes

- Melodía 1 → Estrellita
- Melodía 2 → Himno de la alegría
- Melodía 3 → Feliz cumpleaños

5.1 UART

Una vez cargado el programa, si se conecta la placa a la computadora, se podrán enviar comandos por medio del serial monitor de por ejemplo arduino IDE para tocar el piano.

Cada comando significa una nota, a continuación se podrán apreciar las correspondientes teclas:

- | | | |
|----------|-----------|------------|
| • Do → a | • Sol → g | • Re# → e |
| • Re → s | • La → h | • Fa# → t |
| • Mi → d | • Si → j | • Sol# → y |
| • Fa → f | • Do# → w | • La# → u |

Por medio del UART, se puede enviar un total de cuatro teclas a la vez como máximo, sin embargo, se recomienda enviar de a una para asegurarse el buen funcionamiento.

Anexo

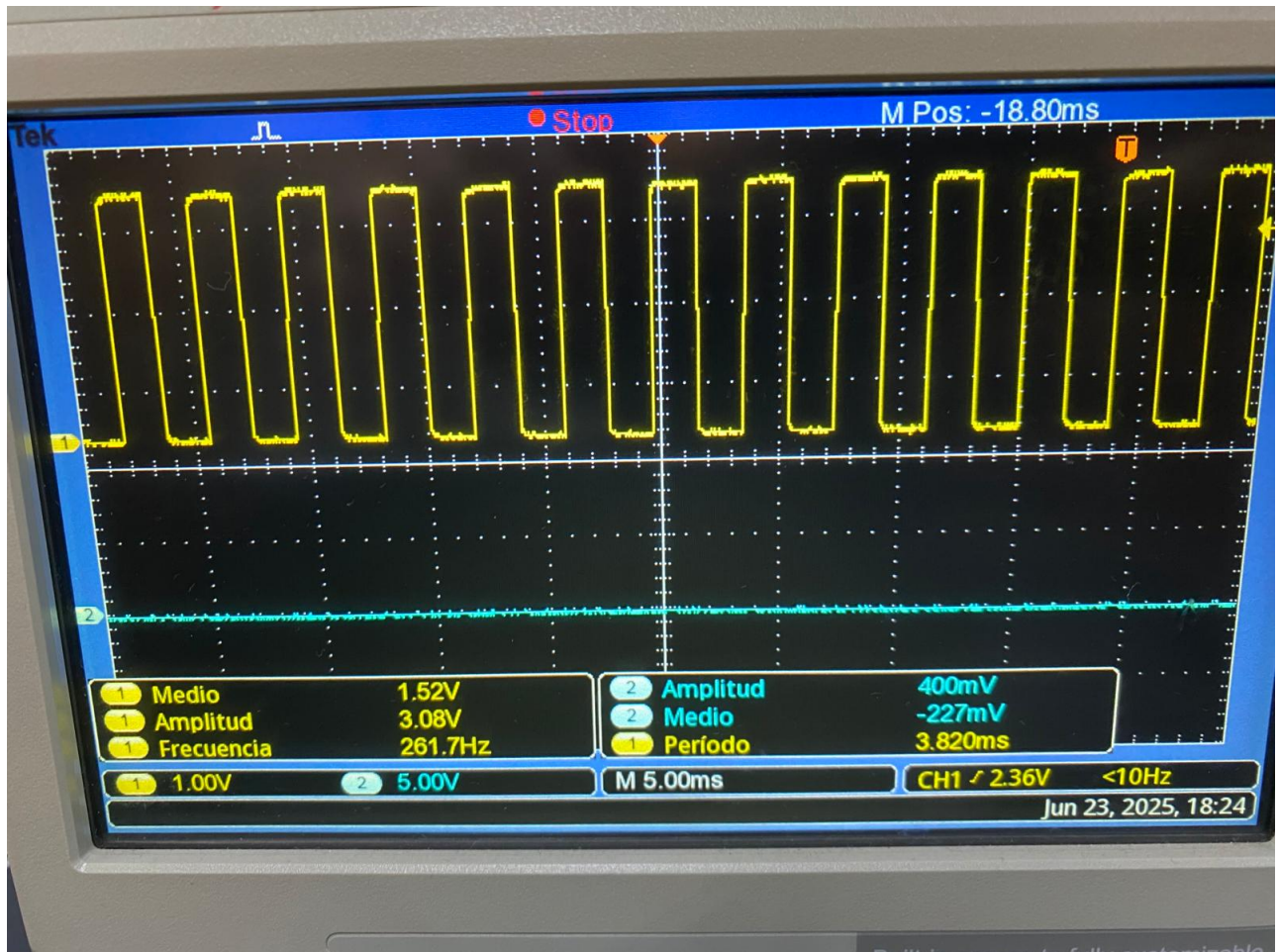


Figure 6.1: Señal generada para nota DO

6.1 Datasheets

Las datasheets utilizadas fueron las siguientes.

- Linker and utilities.pdf
- 10-bit Analog-to-Digital Converter (ADC).pdf
- UART.pdf

- [Timers.pdf](#)
- [Output Compare.pdf](#)
- [I/O Ports.pdf](#)
- [PIC32MX3XX Datasheet.pdf](#)
- [SPI.pdf](#)
- [SSD1306.PDF](#)
- [Flash Programming.pdf](#)